Design, Implementation and Validation of an Object-Oriented Campus Smart-Parking System

Programming in Java | Assignment 1 | CO1

STUDENT	Emad Farooqui
REGISTER NUMBER	REPLACE_WITH_REGISTER_NUMBER
JAVA VERSION	Microsoft OpenJDK 17.0.20
GITHUB	TO_BE_ADDED_AFTER_REPOSITORY_IS_PROVIDED

Repository-ready submission with completed code exercises, full implementation, test evidence, UML, pseudocode and complexity analysis.

Executive summary

This submission analyses, implements, tests and re-evaluates a campus parking system. The original design uses encapsulated domain objects, a jagged array of slots, hierarchical/multilevel inheritance, runtime polymorphism, an abstract `Vehicle`, and an optional `Chargeable` interface. All required supplied programs were completed without changing their `main` behaviour. The complete system passed its normal, boundary and invalid-case tests. For the changed requirements, the design was modified to compose vehicles with a `PricingRule`, model passes through a `Pass` interface, retain the jagged array for five unequal floors, and replace linear ticket lookup with a hash map while preserving EV charging and student/staff users.

Task 1 - Understand the problem and identify the objects

The system receives floor capacities, registration and vehicle type, user category/pass status, entry/exit times, energy delivered to an EV, and entry/exit requests. It produces a deterministic slot allocation or full state, an immutable unique ticket, a fee and receipt, released-slot state, occupancy, and total collection.

Candidate nouns and their justified responsibilities are:

Class	Responsibility
`Vehicle`	Canonical registration identity, common fee API, equality and validation
`TwoWheeler`, `Car`, `ElectricCar`	Vehicle-specific rates; EV adds charging capability
`User`, `Student`, `Staff`, `Visitor`	User identity and pass eligibility
`ParkingFacility`	Own and protect floors/slots; allocate/release/validate
`ParkingTicket`	Permanently identify one entry and connect vehicle, user, slot and time
`ParkingService`	Coordinate entry, duplicate/full checks, exit, billing and collection
`Receipt`	Return an auditable exit result

Verbs become the methods `allocate`, `release`, `enter`, `exit`, `calculateFee`, and `chargingFee`. Rules: a slot cannot contain two vehicles; one registration cannot have two active tickets; numbers are never reused; the first compatible free slot wins; negative time/energy is invalid; every started hour is billed; zero minutes is free; valid staff/student monthly passes are free; and charging is billed only for a fee-paying EV. Campus-local time is assumed.

Parallel arrays (`regNo[]`, `type[]`, `entryTime[]`) also require `O(n)` lookup and `O(n)` total storage, but an attribute requires another synchronised array and every operation must preserve matching indices. Objects keep each vehicle coherent and enforce invariants. They add object-header/reference overhead but are substantially safer and extensible. Full evidence: `Task01/README.md`.

Task 2 - Choose the data representation

Slots, floors, minutes and kWh are `int`; ticket numbers are `long`; registration is a canonical uppercase `String`; entry is `LocalDateTime`; pass state is `boolean`; and rates/fees are `double` to match the brief (production currency should use integer paise or `BigDecimal`).

`Vehicle[][]` is a jagged array: floor lookup and slot lookup are each direct `O(1)`, total storage is `O(total slots)`, and unequal floors are represented without wasted cells. A flattened array also offers `O(1)` lookup but needs a floor-offset table for unequal rows. `ArrayList<Vehicle[]>` supports appended floors at the cost of list

capacity/indirection, unnecessary for a fixed facility.

Integer division risk: `61 / 60 * 30` gives Rs 30 because the fraction is discarded; casting the result later cannot restore it. `(61 + 59) / 60 * 30` correctly gives Rs 60. Validation precedes arithmetic. Full evidence: `Task02/README.md`.

Task 3 - Design the control logic

Slot allocation pseudocode

```
ALGORITHM Allocate(vehicle)
FOR floor <- 0 TO floorCount - 1 DO
  FOR slot <- 0 TO slotsOnFloor(floor) - 1 DO
    IF slots[floor][slot] = NULL THEN
      slots[floor][slot] <- vehicle
      RETURN [floor, slot]
    END IF
  END FOR
END FOR
RETURN NULL
END ALGORITHM
```

Fee pseudocode

```
ALGORITHM ComputeFee(vehicle, user, minutes, kWh)
IF minutes < 0 OR kWh < 0 THEN RAISE invalid input
IF user has valid pass THEN RETURN 0
hours <- (minutes + 59) DIV 60
fee <- hours * vehicle.ratePerHour()
IF vehicle supports Chargeable THEN fee <- fee + vehicle.chargingFee(kWh)
RETURN fee
END ALGORITHM
```

`return` avoids scanning after the first free slot. `continue` would skip an incompatible reserved slot; `break` only exits one loop. If no slot is found, the service reports full capacity. Scanning is worst-case `O(F*S)` allocation and `O(1)` release. Free counts plus stacks make both `O(1)` amortised but need `O(F*S + F)` indexing space and careful ordering to preserve first-fit. The completed `Task03/ParkingFacility.java` exactly matches the expected six output lines.

Task 4 - Classes, objects and constructors

Constructors establish valid objects immediately. `Vehicle` requires registration. A ticket's four-argument constructor chains with `this` to a five-argument form that accepts time, avoiding repeated initialisation; subclass constructors call `super`. No public default constructor exists because partially initialised domain objects are invalid. A copy constructor is not used: a reprinted/duplicate receipt must retain the same immutable ticket identity, not create a new logical ticket.

References/parameters occupy stack frames while objects/arrays occupy heap storage. Java passes references by value. Constructor, setter, and factory creation are each `O(1)` per object, hence `O(n)` time and heap space for `n` tickets. Setters permit an invalid intermediate state; factories are useful for subtype selection; validated constructors are the direct default. Full evidence: `Task04/README.md`.

Task 5 - Static members and object lifecycle

The ticket sequence is a static `AtomicLong`, so uniqueness spans facilities and concurrent gates. Occupancy is per-facility instance state. A facility-owned counter would use `O(k)` words for `k` facilities and could produce duplicate ticket numbers; the static counter uses `O(1)` space regardless of `n` tickets. If numbering were facility-local, the key must include facility ID.

On exit, the active array entry is removed and nulled. When callers and workflows retain no reference, the ticket becomes eligible (not guaranteed immediately) for garbage collection. Full evidence: `Task05/README.md`.

Task 6 - Inheritance hierarchy

```
Vehicle
|-- TwoWheeler
`-- Car
    `-- ElectricCar implements Chargeable

User
|-- Student
|-- Staff
`-- Visitor
```

This is hierarchical plus multilevel inheritance. Registration validation, fee API and identity operations are inherited; subtypes add rate/capability. For `new ElectricCar(...)`, Java runs `Vehicle` initialisation/constructor, then `Car`, then `ElectricCar`. `super` establishes inherited state before subclass code.

Composition with a `VehicleType`/policy makes new rates data-oriented, keeps fee calls `O(1)`, but adds a strategy reference/object. Task 12 selects this trade-off for complex changing pricing. UML source is in `Task06/README.md`.

Task 7 - Override and specialise behaviour

Subclasses override `ratePerHour()` and EV implements `chargingFee()`. `@Override` makes signature/return/access errors compile-time failures. Identity fields are final; final equality/hash methods prevent subtypes from violating collection contracts. `toString`, `equals`, and `hashCode` use canonical registration. Consequently a car and electric car with the same registration compare equal: this is correct because registration represents one physical vehicle and duplicate active registrations are forbidden.

Dynamic dispatch is `O(1)` and new simple behaviour is local to a subtype. A type switch is also commonly `O(1)` in Java, but its central method must be edited for new categories; an if-chain can be `O(k)`. Full evidence: `Task07/README.md`.

Task 8 - Apply polymorphism

Overloaded `calculateFee(int)` and `calculateFee(LocalTime, LocalTime)` demonstrate compile-time selection. A `Vehicle[]` stores different actual subtypes by upcasting. Trace for EV: declared `Vehicle` -> actual `ElectricCar` -> shared `calculateFee(130)` -> dynamically dispatched car rate -> Rs 90 -> `Chargeable` adds 10 x Rs 12 -> Rs 210. This avoids repeated long type tests. `Task08/FeeDemo.java` matches its four expected lines. Full evidence: `Task08/README.md`.

Task 9 - Introduce abstraction

`Vehicle` is abstract because it has state, constructor invariants and shared behaviour but no universal rate. `Chargeable` is an interface because it is an independent optional capability and Java permits multiple interface behaviours. Reversing them loses abstract-class constructors/state for Vehicle and prevents ElectricCar from extending Car while extending an abstract Chargeable class. Both method calls remain `O(1)`; the chosen model

best expresses stateful identity plus orthogonal capabilities. Full evidence: `Task09/README.md`.

Task 10 - Packages and encapsulation

`parking.model` contains the domain, `parking.service` contains facility/use-case logic, and `parking.app` is the executable boundary. Fields are private; cross-package API is public; subclass hooks are protected. Invalid duration, energy, identity, capacity, duplicate registration and positions are rejected at controlled boundaries.

Collections use wrappers/autoboxing when primitive values are stored. `Integer` must be compared with `.equals` or by unboxing, because `==` can compare wrapper identity; unboxing null fails. Caller-side validation is still `O(1)` each time but repeats across `m` callers and can be omitted. Encapsulation centralises it once. Full evidence: `Task10/README.md`.

Task 11 - Implement, test and defend

The complete code is under `Task11/src`. The executed test table is:

Test	Expected	Actual	Result
Two-wheeler, 130 min	30	30	PASS
Exact boundary, 60 min car	30	30	PASS
Zero-minute car	0	0	PASS
EV, 130 min + 10 kWh	210	210	PASS
Hand total collection	270	270	PASS
First position	0:0	0:0	PASS
Last free slot	1:0	1:0	PASS
Full facility	exception	exception	PASS
Duplicate registration	exception	exception	PASS
Monthly pass	0	0	PASS
Unknown exit registration	exception	exception	PASS
Unknown vehicle type	exception	exception	PASS
Invalid slot release	false	false	PASS

Hand total: 3 x 10 + 1 x 30 + 0 + (3 x 30 + 10 x 12) = Rs 270. The design follows all four requested OO principles. Its limitation is the fixed active-ticket array and `O(n)` exit search. `ArrayList` is also `O(n)` search and possibly `O(n)` removal; `HashMap<String, ParkingTicket>` is expected `O(1)` search/removal with `O(n)` storage and greater entry overhead. Task 12 uses the map.

Task 12 - Re-evaluate and adapt the design

Changed facts are two vehicle categories, time-dependent/flat pricing, a visitor pass, and five unequal floors. The original jagged allocation needs only `{2,3,1,4,2}`; its algorithm remains valid. Subclass-only pricing would spread evolving tariff logic, so the system is MODIFIED: `Vehicle` composes a `PricingRule`; hourly, flat and delivery-van rules are independent; `Pass`/`OneDayPass` encapsulate eligibility; and active tickets use a map.

Existing concepts changed were Vehicle, simple vehicle constructors, User and ParkingService. ParkingFacility, allocation and ticket position concepts stayed unchanged. A new simple vehicle needs one subclass selecting an existing rule; a new tariff needs one rule. Calls stay `O(1)` and hash lookup is expected `O(1)`. The extra strategy reference/classes are the trade-off.

Adapted test	Expected	Actual	Result
E-bike 125 min flat fee	25	25	PASS
Delivery van peak, 120 min	120	120	PASS
Delivery van off-peak, 120 min	60	60	PASS
Delivery van crossing 5 PM	90	90	PASS
Valid visitor one-day pass	0	0	PASS
Original EV case retained	210	210	PASS

These results show that modification meets the new requirements while preserving tested allocation logic. Full code/evidence: `Task12/README.md` and `Task12/src`.

Final conclusion

The original design is correct for the original rules and verified by executable tests. The adapted design localises volatile pricing and pass policy while retaining stable identity, ticketing and jagged allocation. It is easier to extend, remains constant-time per fee calculation, and improves expected exit lookup from linear to constant time. The stated limitation is that floating-point currency is used to match the supplied API; production code should migrate amounts to paise or `BigDecimal`.

Appendix - Verified execution evidence

The following screenshots were rendered from the exact console output captured after successful compilation and execution on Java 17.

Task 3 - completed ParkingFacility program

Task 03 - ParkingFacility

```
[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]
```

Task 8 - completed FeeDemo hierarchy

Task 08 - FeeDemo

```
TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0
true
```

Task 11 - original system: 13 passed, 0 failed

Task 11 - Original system tests

TEST	EXPECTED	ACTUAL	RESULT
two-wheeler 130 minutes	30.0	30.0	PASS
exact boundary 60 minutes	30.0	30.0	PASS
zero-minute parking	0.0	0.0	PASS
EV 130 minutes plus 10 kWh	210.0	210.0	PASS
hand total collection	270.0	270.0	PASS
first allocated position	0:0	0:0	PASS
last free slot	1:0	1:0	PASS
full facility	exception	exception	PASS
duplicate registration	exception	exception	PASS
monthly pass is free	0.0	0.0	PASS
invalid registration on exit	exception	exception	PASS
unknown vehicle type	exception	exception	PASS
invalid slot release	false	false	PASS

SUMMARY: 13 passed, 0 failed

Task 12 - adapted system: 6 passed, 0 failed

```
Task 12 - Adapted system tests

TEST                              | EXPECTED | ACTUAL   | RESULT
------------------------------------------------------------------
e-bike flat fee                   |    25.00 |    25.00 | PASS
delivery van peak double rate     |   120.00 |   120.00 | PASS
delivery van off-peak car rate    |    60.00 |    60.00 | PASS
delivery van crosses 5 PM         |    90.00 |    90.00 | PASS
valid visitor one-day pass        |     0.00 |     0.00 | PASS
original EV charging retained     |   210.00 |   210.00 | PASS
SUMMARY: 6 passed, 0 failed
```